



Consul Colect

Dois caminhos para producao

Como por o sistema a funcionar a serio, com duas escolhas de ferramentas — uma com servicos geridos e paga, outra com software livre num servidor proprio — e como o usar depois de estar de pe, com uma brigada a serio.

Antes de escolher

Os dois cenários entregam **o mesmo sistema**. O que muda é quem administra as peças, e quanto se paga por isso.

	Cenário A — gerido	Cenário B — próprio
Para quem	quem quer arrancar em dois dias e não quer administrar servidores	quem tem quem administre, e quer o custo mais baixo
Custo mensal	90 a 170 USD	20 a 35 USD
Tempo até estar de pé	meio dia	um a dois dias
Quem trata de backups	o fornecedor	tu, com um cron
Quem trata de actualizações do sistema operativo	o fornecedor	tu
Quando falha às 3 da manhã	há suporte para abrir um bilhete	há-de haver alguém
Sair para outro sítio	fácil: é tudo Docker	trivial: já é um <code>docker compose</code>

A recomendação honesta: se é o primeiro piloto e ninguém na equipa já administrou um servidor Linux, começa pelo **A**. Passar de A para B mais tarde é um dia de trabalho, porque o que corre nos dois é a mesma imagem Docker. Começar por B sem quem o mantenha é como se descobre, três meses depois, que os backups nunca correram.

O que NÃO muda entre os dois, e é o que interessa: os técnicos gravam sempre primeiro no telefone. Se o servidor estiver em baixo, o trabalho de campo continua e sincroniza depois. A escolha de alojamento não põe em risco nenhum registo.

Cenário A — ferramentas geridas

Tudo pago, tudo administrado por outros. É a escolha de quem prefere gastar dinheiro em vez de tempo.

A.1 O que se contrata

Peça	Serviço	Plano	Custo/mês
Base de dados	Neon ou Supabase (Postgres com PostGIS)	Pro	25–35 USD
API e painel	Railway	Hobby/Pro	20–40 USD
Sincronização	PowerSync Cloud	Starter	35–50 USD
Anexos e mapas	Cloudflare R2	pago ao uso	3–15 USD
Identidade	Keycloak no Railway ou Auth0	—	0–25 USD
Vigilância	Better Stack	Free/Pro	0–20 USD
Erros	Sentry	Team	0–26 USD

Porquê R2 e não S3 da Amazon. O R2 não cobra saída de dados. Numa plataforma em que os telefones descarregam mapas de 300 MB e o escritório exporta fotografias às centenas, a saída é a fatia que cresce sem se dar por ela. No S3 a mesma utilização custa várias vezes mais.

Porquê o Postgres tem de ter PostGIS. As vistas que o QGIS consome são `geometry(Point,4326)`. Um Postgres sem PostGIS arranca, aceita as migrações até certo ponto, e falha ao criar as vistas — com um erro que parece um erro de SQL e é uma extensão em falta.

A.2 Deploy, passo a passo

1. **Base de dados.** Cria o projecto no Neon, activa a extensão PostGIS:

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

Guarda a ligação. Precisas de **duas**: a de administração (dona das tabelas, para as migrações) e a da aplicação.

2. **Migrações.** Do teu computador, uma vez:

```
DATABASE_URL='<ligação de administração>' pnpm db:migrate
DATABASE_URL='<ligação de administração>' pnpm db:seed
```

3. **O papel da aplicação.** Ainda na ligação de administração:

```
ALTER ROLE cvforms_app LOGIN PASSWORD '<uma senha gerada>';
```

Isto não é opcional e não é um detalhe. A API tem de correr com um papel que não seja dono das tabelas e não tenha `BYPASSRLS`. Se correr como dono, o isolamento entre organizações desaparece — e desaparece sem erro nenhum. Um cliente passa a ver os dados de outro e ninguém dá por isso até alguém reparar.

Confirma:

```
SELECT rolsuper, rolbypassrls FROM pg_roles WHERE rolname = 'cvforms_app';  
-- os dois têm de vir `f`
```

4. Keycloak. No Railway, a partir da imagem `quay.io/keycloak/keycloak`, com um volume. Importa `infra/keycloak/`. Cria os clientes `consul-mobile` (público, com PKCE) e `consul-admin` (confidencial).

5. API. No Railway, a partir do repositório, com

`infra/docker/Dockerfile.api`. Variáveis:

<code>DATABASE_URL</code>	a ligação da APLICAÇÃO, não a de administração
<code>KEYCLOAK_ISSUER</code>	<code>https://auth.../realms/cvforms</code>
<code>CORS_ORIGINS</code>	<code>https://painel...</code>
<code>S3_ENDPOINT</code>	<code>https://<conta>.r2.cloudflarestorage.com</code>
<code>S3_PUBLIC_ENDPOINT</code>	o mesmo, ou o domínio próprio do R2
<code>S3_BUCKET</code>	<code>consul-anexos</code>
<code>S3_BUCKET_MAPAS</code>	<code>consul-mapas</code>
<code>POWERSYNC_URL</code>	<code>https://<instância>.powersync.journeyapps.com</code>
<code>SENTRY_DSN</code>	opcional

Healthcheck do serviço: `/health`.

6. Painel. No mesmo projecto, com `infra/docker/Dockerfile.admin`.

<code>API_URL</code>	<code>http://api.railway.internal:4000</code>
<code>NEXTAUTH_URL</code>	<code>https://painel...</code>
<code>NEXTAUTH_SECRET</code>	gerado
<code>KEYCLOAK_*</code>	o realm e o cliente confidencial

7. PowerSync Cloud. Liga à base, cola o conteúdo de

`infra/powersync/sync-rules.yaml`, e aponta o JWKS ao Keycloak.

8. A app. Ver a secção **A app dos técnicos**, mais abaixo — é igual nos dois cenários.

A.3 O que custa mais do que se espera

- O PowerSync Cloud cobra por operações sincronizadas. Com trinta técnicos a fazer trinta registos por dia, o plano Starter chega. Com trezentos, não.
- O plano gratuito do Neon suspende a base quando não há uso. O primeiro

pedido depois de umas horas leva vários segundos — e um técnico com rede má lê isso como avaria. No plano Pro não acontece.

- **O Railway cobra por memória usada, não reservada.** A API em repouso são ~200 MB; o Keycloak são ~600 MB e é a peça mais cara do projecto.

Cenário B — servidor próprio, ferramentas livres

Um servidor, `docker` `compose`, e software livre em tudo. Paga-se o servidor e o domínio; o resto é trabalho.

B.1 O que se contrata

Peça	O que é	Custo/mês
Servidor	Hetzner CX32 (4 vCPU, 8 GB, 80 GB) ou Contabo equivalente	14–18 USD
Cópias de segurança	snapshots do fornecedor	3–5 USD
Domínio	<code>.ao</code> ou <code>.com</code>	~1–3 USD
Postgres + PostGIS	no servidor	0
Keycloak	no servidor	0
PowerSync	Open Edition, no servidor	0
MinIO	no servidor	0
HTTPS	Caddy + Let's Encrypt	0
Vigilância	UptimeRobot (plano gratuito)	0
Erros	Sentry self-hosted, ou nenhum	0

Quando é que se paga alguma coisa neste cenário. Quando os anexos passarem de umas dezenas de GB: aí compensa mais um R2 do que discos no servidor. Muda-se as variáveis `S3_*` e não se muda mais nada — o código já fala S3.

B.2 Deploy, passo a passo

1. O servidor. Ubuntu 24.04, Docker instalado. Fecha tudo menos o essencial:

```
ufw allow 22/tcp && ufw allow 80/tcp && ufw allow 443/tcp && ufw enable
```

2. Os cinco nomes de DNS, a apontar para o IP do servidor, antes de arrancar:

```
paine1.teudominio.ao      api.teudominio.ao      sync.teudominio.ao
auth.teudominio.ao      ficheiros.teudominio.ao
```

Cria-os primeiro. O Caddy pede os certificados ao primeiro arranque; se um nome ainda não resolver, a Let's Encrypt recusa e o Caddy entra em espera exponencial. Parece que o serviço está em baixo, e não está — está à espera de uma hora que já passou.

3. Configuração:

```
git clone <repositório> consul-colect && cd consul-colect
cp .env.producao.example .env.producao

for n in POSTGRES_PASSWORD APP_DB_PASSWORD KEYCLOAK_ADMIN_PASSWORD \
    NEXTAUTH_SECRET POWERSYNC_ADMIN_TOKEN MINIO_ROOT_PASSWORD; do
    echo "$n=$(openssl rand -base64 32 | tr -d '=/+' | cut -c1-32)"
done
```

Cola cada valor no seu sítio, e os cinco domínios nas variáveis `DOMINIO_*`.

4. Arrancar:

```
COMPOSE="docker compose -f infra/docker/docker-compose.prod.yml --env-file .env.producao"

$COMPOSE build
$COMPOSE up -d postgres
$COMPOSE run --rm migracoes

set -a; . ./env.producao; set +a
sh infra/scripts/criar-papel-app.sh

$COMPOSE up -d
$COMPOSE logs -f caddy
```

O script do papel confirma sozinho que ele não é superutilizador nem tem

`BYPASSRSL` — se for, recusa-se a continuar.

5. Keycloak. Em `https://auth....`, com as credenciais do `.env.producao`.

Copia o segredo do cliente `consul-admin` para `KEYCLOAK_ADMIN_CLIENT_SECRET`

e reinicia o painel.

6. Confirmar:

```
curl -s https://api.teudominio.ao/health
```

Os quatro `dependencias` a `true`.

7. Cópias de segurança, no cron:

```
0 3 * * * cd /opt/consul-colect && docker compose -f infra/docker/docker-compose.prod.yml \
    --env-file .env.producao exec -T postgres pg_dump -U cvforms cvforms \
    | gzip > /backups/consul-$(date +%F).sql.gz
0 4 * * * find /backups -name 'consul-*.sql.gz' -mtime +30 -delete
```

Restaura um backup uma vez, esta semana, antes de precisares. Um backup que nunca foi restaurado não é um backup — é um ficheiro que se espera que sirva.

8. Vigilância. No UptimeRobot, dois monitores:

- `https://api.teudominio.ao/health` — está de pé?
- `https://api.teudominio.ao/health/replicacao` — o mais importante

O segundo responde **503** quando um slot de replicação está a acumular WAL.

Sem esse monitor, esse 503 não chega a ninguém — e é o único aviso que existe antes de o disco encher e o Postgres parar de aceitar registos de campo.

B.3 O que corre mal neste cenário

- **Ninguém actualiza o sistema operativo.** Põe `unattended-upgrades` .
- **O disco enche com fotografias.** Vigia com `df -h` no mesmo cron dos

backups. Quando passar de 70 %, muda os anexos para R2.

- **O Keycloak é a peça que mais memória gasta.** Num servidor de 4 GB fica apertado com tudo o resto; 8 GB é o mínimo confortável.

A app dos técnicos

Igual nos dois cenários. O servidor entra no pacote, na compilação. O

técnico nunca vê um URL nem escreve um endereço — o que ele sabe é o utilizador e a senha.

```
cd apps/mobile
cat > .env.production <<'FIM'
EXPO_PUBLIC_API_URL=https://api.teudominio.ao
EXPO_PUBLIC_KEYCLOAK_ISSUER=https://auth.teudominio.ao/realms/cvforms
EXPO_PUBLIC_KEYCLOAK_CLIENT_ID=consul-mobile
EXPO_PUBLIC_POWERSYNC_URL=https://sync.teudominio.ao
FIM

node ../../node_modules/expo/bin/cli prebuild --platform android --clean
cd android && ./gradlew assembleRelease
```

Tudo o que começa por `EXPO_PUBLIC_` fica dentro do pacote e é legível por quem o descompilar. URLs podem lá estar; senhas e segredos, nunca. O cliente OIDC é público e usa PKCE precisamente para não precisar de nenhum.

Distribui o `.apk` por link ou por cabo. Para actualizar, gera um APK novo com a versão subida.

Antes de mandar alguém actualizar, confirma que a lista de «por enviar» está a zero nesse telefone. Os registos sobrevivem a uma actualização, mas não vale a pena arriscar por um minuto de espera.

Usar o sistema depois de estar de pé

Esta secção é o guião de um piloto a sério. Segue por ordem.

Semana 0 — o escritório sozinho

1. **Criar as pessoas.** No Keycloak, um utilizador por técnico. A linha na base do Consul Colect aparece sozinha ao primeiro login — não se criam pessoas no painel, e é de propósito: a identidade vive num sítio só.
2. **Desenhar o primeiro formulário.** No painel, em **Formulários**. Começa com **seis a oito perguntas**, não com quarenta. Um formulário grande à primeira esconde os problemas de desenho por baixo do cansaço de o preencher. Põe um campo de geometria, um de fotografia, e um campo de texto marcado como **pesquisável** — é o que o técnico vai usar para saber se já passou por um sítio.
3. **Publicar.** Só depois de publicado é que o formulário existe para a app.
4. **Atribuir.** Em **Atribuições**, dá acesso a ti próprio primeiro. Se quiseres experimentar âmbitos, acrescenta um campo `município`, marca-o como campo de âmbito **antes de haver registos**, e dá a um técnico o filtro de um município só.
5. **Carregar um mapa.** Em **Mapas**, um `.pmtiles` da área do piloto. Sem ele, o mapa da app fica cinzento sem rede.

Semana 1 — dois telefones, uma pessoa

Faz isto tu, antes de envolver a brigada.

O que fazer	O que tem de acontecer
Instalar o APK e entrar	os formulários aparecem em segundos
Modo de avião, preencher, fechar a app à força, reabrir	o rascunho está intacto
Obter posição ao ar livre	coordenada, precisão em metros, <code>single</code> , <code>internal</code>
Obter posição junto a um muro	aviso laranja e caixa de justificação
Tirar uma fotografia	entra reduzida, e o registo sincroniza mesmo com ela por subir
Submeter em modo de avião	fica em Por enviar
Desligar o modo de avião	passa a Enviados sozinho
Abrir o Mapa sem rede	o mosaico desenha-se com os pontos por cima
Editar o mesmo registo em dois telefones , ambos offline, e depois ligar	as duas versões ficam; o registo vai para Por rever no painel
Resolver no painel	escolhes uma, escreves porquê, e a outra continua no histórico
Exportar CSV e abrir no Excel	cabeçalhos com os rótulos das perguntas, acentos certos
Abrir o GeoJSON no QGIS	os pontos no sítio, com <code>_accuracy_m</code> e <code>_fix_type</code>

Se qualquer uma destas falhar, pára aqui. É muito mais barato corrigir antes de haver trinta pessoas a usar.

Semana 2 — a brigada

1. Uma manhã de formação, com telefones na mão. Não é uma apresentação.

Cada pessoa faz três registos a sério, com o formador ao lado. Duas horas chegam.

2. Entrega o manual (`docs/MANUAL-DO-TECNICO.md`) impresso, e o cartão de bolso recortado.

3. Os três hábitos que valem mais do que o resto do manual:

- **Na dúvida, grava.** Um registo repetido resolve-se no escritório. Uma visita perdida obriga alguém a voltar lá.
- **Nunca entregar um telefone com 🕒 na lista.** É a única forma conhecida de perder trabalho neste sistema.
- **Quando o aviso laranja aparecer, escrever porquê a sério.** «sem sinal» não

ajuda ninguém; «encostado ao muro do PT» ajuda.

4. **Primeiro dia de campo com uma equipa só**, não com todas. Ao fim do dia, no painel: quantos registos chegaram, quantos ficaram por rever, e o relatório de qualidade.

O que olhar todas as semanas

Onde	O que procurar
Painel → Por rever	se cresce sem parar, os âmbitos estão largos de mais ou dois técnicos andam na mesma área
Exportações → relatório de qualidade	pontos acima do limiar sem justificação escrita — é um hábito a corrigir com pessoas, não com código
<code>/health/replicacao</code>	tem de dar 200
Espaço em disco	acima de 70 %, decide o que fazer aos anexos

Sinais de que alguma coisa está mal

Sintoma	Quase sempre é
um técnico não vê nenhum formulário	falta a atribuição
vê uns registos e não outros	tem um filtro de âmbito
os registos ficam em ⌚ dias a fio	o <code>EXPO_PUBLIC_API_URL</code> do APK aponta para outro lado
as fotografias não sobem	o <code>S3_PUBLIC_ENDPOINT</code> não bate certo com o domínio dos ficheiros
o mapa aparece cinzento	não há camada offline descarregada
um técnico vê dados de outra organização	pára tudo. A API está a correr como dono das tabelas

A última é a única que é uma emergência. As outras esperam pela manhã seguinte.

Como relatar um problema

O mínimo útil: o que estavas a fazer, o que esperavas, o que aconteceu, e onde — painel, app ou API. Se for na app, o modelo do telefone e se tinhas rede.

```
curl -s https://api.teudominio.ao/health      # estado dos serviços
adb logcat -d | grep -i consul | tail -50     # o telefone, por cabo
```

Se a resposta trouxe um `request_id`, manda-o: é o que liga o erro que viste à linha do log.

Nunca mandes senhas nem o conteúdo do `.env.producao`. Se um log trazer um URL de base de dados, apaga a parte da senha antes de o colar.

Se um registo desapareceu, não mexas em nada e diz logo. As revisões são append-only e quase de certeza o registo está lá — o que falhou foi mostrá-lo.